

notes-world — Architecture (Current State)

Snapshot date: 2026-06-17

Status: current

Author: generated from a verified codebase sweep (not from prior docs)

Live: <https://notes-world.christopherrehm.de>

1. System Overview

notes-world is a single-user-per-account personal productivity app — notes, ideas, tasks, reminders, checklists, and tags, organised on a configurable dashboard. It is in daily personal use and runs as a Docker Compose stack on a VPS behind a host Apache TLS proxy.

It ships through **three client surfaces** over **one REST API**:

- a **React 18 web SPA** (the primary UI),
- an **Expo / React Native mobile app** (Android, Play closed-testing), and
- an **MCP server** that lets AI agents (Claude Code, Claude Desktop, claude.ai connector) read and write the same data.

All three talk to the same Express API and PostgreSQL 16 database. There is no second source of truth — the mobile app and the MCP server are thin clients of the REST API.

Layer / component diagram



2. Monorepo / Workspace Layout

Yarn-workspaces monorepo (npm-driven scripts; lockfile `package-lock.json`). Node 20 throughout.

```
packages/
  server/   Express 4 REST API (TypeScript)    - port 3001
  web/      React 18 SPA (Vite, Tailwind)      - port 5173 (dev)
  shared/   Shared TS types + pure utils (zod)
  mcp/      MCP server (HTTP transport)        - port 3002
  mobile/   Expo SDK 54 / React Native app
  importer/ Python markdown bulk-importer (CLI)

src-desktop/ Legacy single-package structure - reference only, do not edit
docs/        Project documentation (this file lives here)
nginx/       Reverse-proxy config
.github/workflows/ ci.yml, android-release.yml
```

Package	Pkg name	Stack / key deps
server	@notes-world/server@0.1.0	Express 4.18, pg 8.11, jsonwebtoken 9, bcrypt 6, express-rate-limit 8, helmet 7, Stripe 22, archiver 7
web	@notes-world/web@0.1.0	React 18.2, React Router 7.15, Vite 6.4, Tailwind 3.4, i18next 26 / react-i18next 17
shared	@notes-world/shared@0.1.0	zod 3.22 (no React/Node runtime deps — types + date/sort utils)
mcp	@notes-world/mcp@0.1.0	@modelcontextprotocol/sdk 1.29, Express 4.22, pg 8.11, jsonwebtoken 9
mobile	@notes-world/mobile@0.1.0	Expo ~54, Expo Router ~6, React Native 0.81.5, React 19.1, datetimetype 8.4, expo-secure-store, i18next 26

Package	Pkg name	Stack / key deps
importer	(Python)	stdlib + <code>requirements.txt</code> ; modules: walker, parser, classifier, tagger, dedup, inserter, pipeline

The `shared` package is the seam that keeps web and mobile behaviour identical — notably the date-sorting logic (see §6).

3. Web Frontend (`packages/web`)

- **Framework:** React 18.2 + React Router 7.15, built with Vite 6.4.
- **Styling:** Tailwind CSS 3.4 + PostCSS.
- **State:** a global `AppContext` plus custom hooks; data fetched through a thin `api/` fetch wrapper.
- **i18n:** `i18next` + `react-i18next`, 26 locale JSON files (see §8).
- **Tests:** Vitest 4.1 + Testing Library.

Key areas:

```
src/
  components/      TagView.tsx, ItemDrawer.tsx, drawer/{TaskFields,TagPicker,DependenciesPanel}
  pages/           route pages
  context/         AppContext (global state)
  hooks/           custom hooks (incl. sortable-list behaviour)
  api/             fetch wrapper + endpoint calls (in-memory access token)
  i18n/locales/    26 translation files
  types/, utils/   types + helpers (linkify, etc.)
```

Auth flow (web): login returns a short-lived access token held **in memory** in `api/`;

the refresh token is an HTTP-only cookie set by the server. On a 401/403 the client silently calls `/api/auth/refresh` and retries the original request once; if refresh fails the user is logged out.

Recent work — due dates + sort toggle (web):

- **Due-date display/edit:** `ItemDrawer.tsx` (`TaskDates`) renders `<input type="date">` for `due_date` and `start_date`, persisted into the item's `type_data` JSONB.
- **Manual/Date sort toggle:** `TagView.tsx` adds "Due Date" / "Start Date" buttons to the tag header. `sortByDate(field)` calls `sortItemsByDate(items, field)` from `shared`, persists the resulting order via `api.sortOrders.save("tag:<id>", [...ids])`, and bumps a `sortNonce` to remount the sortable list. After a date sort, **manual drag-and-drop remains available** — the date sort simply writes a new manual order, so the two modes share one persisted ordering per context.

4. Mobile App (`packages/mobile`)

- **Runtime:** Expo SDK ~54, React Native 0.81.5, React 19.1.
- **Navigation:** Expo Router ~6 (file-based) with an auth stack and a tab layout.

- **Storage:** access/refresh tokens in `expo-secure-store`; misc state in `AsyncStorage`.
- **i18n:** same 26 locales as web (i18next 26).
- **Tests:** Vitest 4.1.

Navigation shape:

```
app/
  _layout.tsx
  (auth)/ welcome.tsx, login.tsx
  (tabs)/ index(dashboard), tags, checklists, capture, account
  tag/[id].tsx          sortable tag list (due/start date sort + dividers)
  item/[id].tsx         item editor (date picker)
  checklist/[id].tsx, dashboard.tsx, trash.tsx, changelog.tsx
```

API client: `src/api/client.ts` points at <https://notes-world.christopherrehm.de/api>.

Refresh tokens rotate (sent via `X-Refresh-Token`); concurrent refreshes are coalesced into a shared promise; an auth-failure handler drops the user to login.

Recent work — mobile port of due dates + sort toggle:

- `app/tag/[id].tsx` has a sort bar ("Add Divider", "Sort By", "Due Date", "Start Date"). `sortByDate(field)` uses `src/lib/dueDate.ts` (mirrors `shared` date utils) and persists via `saveSortOrder("tag:<id>", [...ids])`. Manual reordering is a header "Reorder" toggle with up/down arrows; dividers can collapse to hide their children.

- `app/item/[id].tsx` edits `due_date` / `start_date` via `@react-native-community/datetimepicker`, writing the same `type_data` shape as web.

This is the mobile counterpart of the web feature in §3 and is the most recent cross-surface change.

5. Backend & Data Layer (`packages/server`)

Domain architecture

Twelve domains, each following a controller → service → repository layering:

`admin`, `auth`, `billing`, `bug-reports`, `client-errors`, `checklists`, `export`, `import`, `items`, `relationships`, `sort-orders`, `views`.

Domains do not import each other directly; they communicate through an in-process typed `eventBus` (`src/events/eventBus.ts`, a Node `EventEmitter` wrapper). Events include `ItemCaptured/Updated/Promoted/Archived/Restored`, `TaskCompleted/Started/Blocked`, `ItemTagged/Untagged`, `TagDeleted`, `Dependency*`, `ImportCompleted`.

API surface (mounted under `/api`): `auth`, `items`, `tags`, `dependencies`, `dashboard`, `import`, `sort-orders`, `export`, `checklists`, `billing` (Stripe webhook), `admin`, `bug-reports`, `client-errors`. A public `/health` endpoint backs the Docker healthcheck.

- **Hardening:** helmet, morgan logging, `trust proxy` set for the Apache→nginx→app hop.
- **Rate limiting:** ~200 req/min global; stricter (~30 / 15 min) on auth endpoints.

- **DB access:** raw SQL via `pg Pool (src/db/client.ts)` — **no ORM**. Migrations run idempotently on startup.

Data model

Migrations are append-only in `src/db/migrations/` (`001–017`; never edit existing files):

```
001 initial schema    002 import schema    003 import pipeline    004 tag source
005 sort orders        006 dividers          007 sort-order index    008 dividers-as-items
009 archived_at        010 tag color          011 item color          012 auth (users, refresh_tokens)
013 billing            014 coupons            015 api keys            016 checklists
017 user disabled
```

Core tables:

- **items** — `id`, `user_id`, `title`, `body`, `item_type` (`Untyped|Task|Idea|Note|Reminder|Divider`), `status` (`Active|Archived`), `type_data` **JSONB**, `color`, `archived_at`, timestamps.

Type-variant fields (`task_status`, `priority`, **`due_date`**, `start_date`, `completed_at`, `maturity`, `category`, `remind_at`, ...) live in `type_data` — promoting an item to a new type adds fields without losing the base record. ****Due dates are `YYYY-MM-DD` strings inside `type_data`, not a dedicated column**** (parsed at runtime for sort/display).

- **tags** — `id`, `user_id`, `name`, `tag_source` (`folder|file|semantic|manual`), `color`, unique (`user_id`, `name`).
- **item_tags** — join table, cascade delete on item and tag.
- **item_sort_orders** — (`user_id`, `context_key`, `item_id`, `sort_order`), unique per (`user_id`, `context_key`, `item_id`). `context_key` is e.g. `tag:<id>` or `maturity:Seed`.

This is what both the manual drag order and the date-sort result write to.

- **users / refresh_tokens** — auth (added in 012); `users.disabled` flag (017).
- **dashboards / blocks** — one dashboard per user (1–4 columns); blocks carry `view_type` and a **JSONB `config`**.
- **checklists / checklist_items** — shopping/checklist feature (016).
- `billing/coupons/api-keys` tables back Stripe subscriptions and `nw_`-prefixed API keys.

Dividers are modelled as items (`item_type=Divider`) since migration 008. All tables carry `user_id` (ADR-005), so multi-tenancy is data-level, not bolted on.

6. Cross-Surface Shared Logic

`packages/shared` holds the types and the **date-sort algorithm** used identically by web and mobile: `sortItemsByDate(items, field)` orders dated items ascending, pushes undated items to the end, and tie-breaks by title. Mobile keeps a thin local mirror (`src/lib/dueDate.ts`) for RN ergonomics. Keeping this in one place is why the web feature and its mobile port behave the same.

7. Authentication & Authorization

- **Scheme:** JWT access tokens + rotating refresh tokens, plus `nw_`-prefixed API keys (SHA-256 hashed at rest) for programmatic/MCP access.
 - **Server:** `requireAuth` middleware reads `Authorization: Bearer <token>`; refresh tokens are stored hashed in `refresh_tokens`; passwords hashed with bcrypt.
 - **Web:** access token in memory, refresh token in an HTTP-only cookie, silent refresh + retry.
 - **Mobile:** both tokens in `expo-secure-store`; refresh via `X-Refresh-Token`, rotated each use.
 - **MCP:** OAuth 2.1 (discovery + authorize + token) for interactive clients, or a `nw_` API key; every tool call requires a valid bearer credential.
 - **Admin:** dedicated admin domain (recent additions: reset user password, enable/disable accounts) gated to admin users.
-

8. Internationalization

i18next (v26) on both web and mobile, with **26 locale files each**, kept in sync:

`bg, cs, da, de, el, en, es, et, fi, fr, ga, hr, hu, it, lt, lv, mt, nl, pl, pt, ro, ru, sk, sl, sv, zh`.

Locales live at `packages/web/src/i18n/locales/*.json` and `packages/mobile/src/i18n/locales/*.json`. New UI strings (e.g. due/start-date labels, the sort buttons, the changelog) must be added to both trees.

9. MCP Server (`packages/mcp`)

Gives AI agents first-class access to the same data as a human user.

- **Transport:** Streamable HTTP (stateless — a fresh `McpServer` per request) on port 3002, exposed at `/mcp` via nginx.
 - **Tools:** item, tag, task, search, export, and checklist tool groups (`@modelcontextprotocol/sdk 1.29`).
 - **Auth:** OAuth 2.1 endpoints (`/.well-known/oauth-authorization-server`, `/oauth/authorize`, `/oauth/token`) for connectors; tool calls authenticate via JWT or `nw_` API key. OAuth state persists to a Docker volume (`mcp_oauth_store`).
 - **Data path:** the MCP server calls the **REST API** (using a `NOTES_WORLD_API_KEY`), not the database directly (ADR-011) — so it stays decoupled from internals.
-

10. Build, Test & Deploy (CI/CD)

Local

```

npm run dev          # web + server
npm run build --workspace=packages/web # SPA build
npm test             # server (Jest) + web/mobile (Vitest)
docker compose up --build # full prod-like stack

```

Server tests run on **Jest 29**; web and mobile on **Vitest 4.1**. Root **npm test** runs all three.

Production stack (`docker-compose.yml`)

Service	Image	Role
app	node:20-alpine (multi-stage)	Express API :3001
mcp	node:20-alpine (multi-stage)	MCP server :3002
nginx	nginx:alpine	reverse proxy (SPA + <code>/api</code> + <code>/mcp</code> + <code>/oauth</code>) on 127.0.0.1:8082
db	postgres:16-alpine	PostgreSQL 16
db-backup	prodrigestivill/postgres-backup-local	daily backup + rotation

TLS terminates at **host Apache (:443)** which forwards to the nginx container. The multi-stage `Dockerfile` builds `shared` once and reuses it for the server and mcp images; the web SPA is built and served statically by nginx.

Pipelines (`.github/workflows/`)

`ci.yml` — **push to master auto-deploys**. It installs deps, builds `shared`, runs server + web + mobile tests, regenerates the changelog, builds the web SPA, then `**rsyncs` the source and `web/dist` to the VPS and runs `docker compose up --build -d**`, finally checking container logs. There is no manual deploy step in the normal flow — merging to master ships it.

`android-release.yml` — **mobile changes ship to Play closed-testing**. Triggered by pushes to `master` touching `packages/mobile/**` (or the workflow file), plus manual `workflow_dispatch`. It guards on the `PLAY_SERVICE_ACCOUNT_JSON` secret, sets up Java 17 + the Android SDK, computes a monotonic `versionCode`, injects version info, builds a release **AAB**, and uploads it to the Google Play **closed-testing (alpha)** track via `scripts/play_upload.py`.

The deploy skill `/deploy-notes-world` automates the same VPS deploy for out-of-band pushes.

11. Risks & Tech Debt

- **Due dates live in JSONB strings**. `type_data.due_date` is a `YYYY-MM-DD` string, so sorting and "overdue" filters parse at runtime and can't use a DB index. Fine at single-user scale; would need a real column/index if date queries grow.
- **One ordering per context**. Manual order and date-sort both write `item_sort_orders`, so a date sort overwrites the user's manual arrangement for that tag. There's no separate "view mode" — intentional today, but surprising if a user expects manual order to be remembered separately.
- **Two date-util implementations**. `shared` (web) and `mobile/src/lib/dueDate.ts` must stay in

lockstep by hand; divergence would silently desync sort behaviour between surfaces.

- **26 locale files × 2 trees, maintained manually.** New strings must land in 52 files; partial translations can ship unnoticed without a key-coverage check.
 - **Migrations run on startup with no down-migrations.** Append-only is safe for forward motion but offers no rollback; a bad migration is recovered via the db-backup volume, not code.
 - **Single-VPS, single-Postgres.** No HA/replica; availability rests on the daily backup + rotation. Acceptable for a personal app, a real risk if it grows.
 - **TLS depends on host Apache** outside the compose file — infra not fully captured in the repo; a host rebuild needs that Apache vhost reconstructed from [docs/deployment.md](#).
 - **Doc drift.** Several overlapping architecture docs already exist ([architecture.md](#), [arch.md](#)); see the recommendation accompanying this file.
-

Appendix — Key paths

- Root: [package.json](#), [docker-compose.yml](#), [Dockerfile](#), [nginx/nginx.conf](#)
- Server: [packages/server/src/{domains,events,db/migrations,middleware}](#)
- Web: [packages/web/src/{components,context,api,i18n/locales}](#)
- Mobile: [packages/mobile/app](#), [packages/mobile/src/{api,lib,i18n/locales}](#)
- MCP: [packages/mcp/src](#)
- CI/CD: [.github/workflows/{ci.yml,android-release.yml}](#), [scripts/play_upload.py](#)